

OBSERVATION TASK 3

Title

UPI Payment and Digital Wallet System using Interface and Exception Handling

Course Outcome: CO3

Objective / Aim

To design and implement a **UPI Payment and Digital Wallet System in Java using interfaces and exception handling**, simulating real-world digital payment operations such as **wallet creation, money addition, UPI payments, balance checking, UPI validation, transaction validation, and exception handling**.

Problem Statement

Design a **digital wallet system** for a user.

The system should allow a user to:

- Create a wallet with a name, mobile number, and UPI ID.
- Add money to the wallet.
- Make a payment using the UPI ID of another user.
- Check the available wallet balance.
- Validate the UPI ID before processing a payment.
- Prevent payments when the balance is insufficient.
- Prevent invalid or negative transaction amounts.
- Handle invalid transactions using appropriate exceptions.
- Display the final wallet details and transaction status.

The program demonstrates:

- Interface
- Implementation
- Encapsulation
- Classes and objects
- Constructors

- Custom exceptions
 - Exception handling
 - `try-catch-finally`
 - String validation
 - UPI payment processing
-

Class and Interface Design

1. PaymentService Interface

The `PaymentService` interface defines the common payment operations:

- `pay()`
- `checkBalance()`

2. Wallet Class

The `Wallet` class stores:

- User name
- Mobile number
- UPI ID
- Balance

The data members are declared `private` to demonstrate **encapsulation**.

Methods:

- `addMoney()`
- `deductMoney()`
- `getBalance()`
- `getUpiId()`
- `displayWalletDetails()`

3. UPIPayment Class

The `UPIPayment` class implements the `PaymentService` interface.

Methods:

- `pay()`
- `checkBalance()`

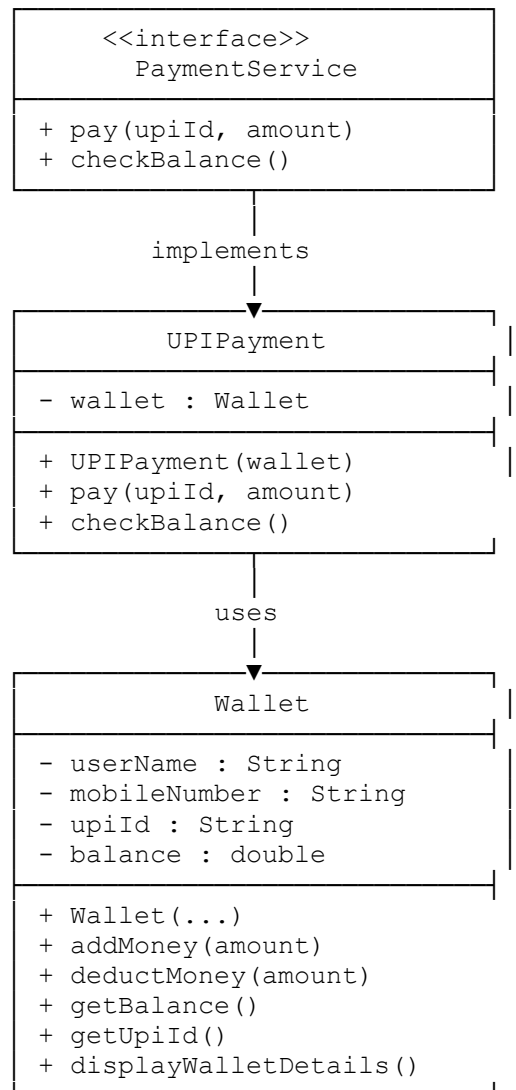
The class uses a `Wallet` object to perform payment and balance operations.

4. Custom Exceptions

Three custom exception classes are created:

- `InsufficientBalanceException`
- `InvalidUPIException`
- `InvalidAmountException`

UML Class Diagram



Custom Exceptions
InsufficientBalanceException InvalidUPIException InvalidAmountException

12 34 Steps to Perform the Task

1. Create an interface named `PaymentService`.
2. Declare `pay()` and `checkBalance()` methods in the interface.
3. Create a `Wallet` class.
4. Declare user information and balance as private data members.
5. Create a constructor to initialize the wallet details.
6. Define `addMoney()` to add money to the wallet.
7. Define `deductMoney()` to deduct money from the wallet after successful payment.
8. Create an `UPIPayment` class that implements `PaymentService`.
9. Create an object of `Wallet` and associate it with the `UPIPayment` object.
10. Validate the UPI ID using String operations.
11. If the UPI ID is invalid, throw `InvalidUPIException`.
12. Validate the payment amount.
13. If the amount is zero or negative, throw `InvalidAmountException`.
14. Check whether sufficient balance is available.
15. If the balance is insufficient, throw `InsufficientBalanceException`.
16. If all validations are successful, deduct the payment amount from the wallet balance.
17. Display the payment success message.
18. Use `try-catch-finally` to handle transaction exceptions.
19. Display the final wallet details.
20. Display the final wallet balance.

Source Code

File: `UPIPaymentDemo.java`

```
package tasks;

// Interface
interface PaymentService {

    void pay(String upiId, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
```

```

        InsufficientBalanceException;

    void checkBalance();
}

// Custom Exception 1
class InsufficientBalanceException extends Exception {

    InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom Exception 2
class InvalidUPIException extends Exception {

    InvalidUPIException(String message) {
        super(message);
    }
}

// Custom Exception 3
class InvalidAmountException extends Exception {

    InvalidAmountException(String message) {
        super(message);
    }
}

// Wallet Class
class Wallet {

    // Private data members - Encapsulation
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    // Constructor
    Wallet(String userName, String mobileNumber,
           String upiId, double balance) {

        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    // Add money to wallet
    void addMoney(double amount) throws InvalidAmountException {

        if (amount <= 0) {

```

```

        throw new InvalidAmountException(
            "Amount must be greater than zero."
        );
    }

    balance = balance + amount;

    System.out.println("Money added successfully: Rs." + amount);
}

// Deduct money from wallet
void deductMoney(double amount) {

    balance = balance - amount;
}

// Get balance
double getBalance() {

    return balance;
}

// Get UPI ID
String getUpiId() {

    return upiId;
}

// Display wallet details
void displayWalletDetails() {

    System.out.println("\n===== WALLET DETAILS =====");
    System.out.println("User Name      : " + userName);
    System.out.println("Mobile Number : " + mobileNumber);
    System.out.println("UPI ID       : " + upiId);
    System.out.println("Balance      : Rs." + balance);
}
}

// UPI Payment Class
class UPIPayment implements PaymentService {

    private Wallet wallet;

    // Constructor
    UPIPayment(Wallet wallet) {

        this.wallet = wallet;
    }

    // Pay method

```

```

@Override
public void pay(String upiId, double amount)
    throws InvalidUPIException,
           InvalidAmountException,
           InsufficientBalanceException {

    // Validate UPI ID using String methods
    if (upiId == null ||
        !upiId.contains("@") ||
        upiId.startsWith("@") ||
        upiId.endsWith("@")) {

        throw new InvalidUPIException(
            "Invalid UPI ID: " + upiId
        );
    }

    // Validate payment amount
    if (amount <= 0) {

        throw new InvalidAmountException(
            "Payment amount must be greater than zero."
        );
    }

    // Check sufficient balance
    if (amount > wallet.getBalance()) {

        throw new InsufficientBalanceException(
            "Insufficient wallet balance."
        );
    }

    // Deduct amount from wallet
    wallet.deductMoney(amount);

    // Payment successful
    System.out.println("Payment successful!");
    System.out.println("Paid to      : " + upiId);
    System.out.println("Amount paid : Rs." + amount);
}

// Check balance
@Override
public void checkBalance() {

    System.out.println(
        "Available Balance: Rs." + wallet.getBalance()
    );
}
}

```

```

// Main Class
public class UPIPaymentDemo {

    public static void main(String[] args) {

        // Create Wallet object
        Wallet wallet = new Wallet(
            "Rahul",
            "9876543210",
            "rahul@upi",
            5000
        );

        // Create UPIPayment object
        PaymentService payment = new UPIPayment(wallet);

        // Display initial wallet details
        wallet.displayWalletDetails();

        // Add money
        System.out.println("\n===== ADD MONEY =====");

        try {

            wallet.addMoney(2000);

        } catch (InvalidAmountException e) {

            System.out.println("Error: " + e.getMessage());

        }

        // Check balance
        System.out.println("\n===== BALANCE =====");

        payment.checkBalance();

        // Transaction 1 - Successful Payment
        System.out.println("\n===== TRANSACTION 1 =====");

        try {

            payment.pay("arun@upi", 2000);

        } catch (InvalidUPIException |
            InvalidAmountException |
            InsufficientBalanceException e) {

            System.out.println(
                "Transaction failed: " + e.getMessage()
            );

        }

    }

}

```

```

    } finally {

        System.out.println("Transaction 1 completed.");
    }

    // Transaction 2 - Invalid UPI
    System.out.println("\n===== TRANSACTION 2 =====");

    try {

        payment.pay("invalidupi", 1000);

    } catch (InvalidUPIException |
            InvalidAmountException |
            InsufficientBalanceException e) {

        System.out.println(
            "Transaction failed: " + e.getMessage()
        );

    } finally {

        System.out.println("Transaction 2 completed.");
    }

    // Transaction 3 - Invalid Amount
    System.out.println("\n===== TRANSACTION 3 =====");

    try {

        payment.pay("priya@upi", -500);

    } catch (InvalidUPIException |
            InvalidAmountException |
            InsufficientBalanceException e) {

        System.out.println(
            "Transaction failed: " + e.getMessage()
        );

    } finally {

        System.out.println("Transaction 3 completed.");
    }

    // Transaction 4 - Insufficient Balance
    System.out.println("\n===== TRANSACTION 4 =====");

    try {

        payment.pay("kiran@upi", 10000);

    } catch (InvalidUPIException |
            InvalidAmountException |

```

```
        InsufficientBalanceException e) {

            System.out.println(
                "Transaction failed: " + e.getMessage()
            );

        } finally {

            System.out.println("Transaction 4 completed.");
        }

        // Display final wallet details
        wallet.displayWalletDetails();

        // Display final balance
        System.out.println("\n===== FINAL BALANCE =====");

        payment.checkBalance();
    }
}
```



Transaction Calculation

Initial Wallet Balance

Initial Balance = Rs.5000

Add Money

5000 + 2000 = Rs.7000

Therefore, the available balance becomes:

Rs.7000

Transaction 1 – Successful Payment

Payment made to arun@upi:

7000 - 2000 = Rs.5000

Remaining balance:

Rs.5000

Transaction 2 – Invalid UPI

UPI ID:

invalidupi

The ID does not contain @, so:

InvalidUPIException

No money is deducted.

Balance remains:

Rs.5000

Transaction 3 – Invalid Amount

Payment amount:

-Rs.500

Since the amount is less than or equal to zero:

InvalidAmountException

No money is deducted.

Balance remains:

Rs.5000

Transaction 4 – Insufficient Balance

Payment amount:

Rs.10000

Available balance:

Rs.5000

Since:

10000 > 5000

the program throws:

InsufficientBalanceException

No money is deducted.

Final Balance

Final Balance = Rs.5000

Sample Output

```
===== WALLET DETAILS =====
```

```
User Name      : Rahul
Mobile Number  : 9876543210
UPI ID        : rahul@upi
Balance       : Rs.5000.0
```

```
===== ADD MONEY =====
```

```
Money added successfully: Rs.2000.0
```

```
===== BALANCE =====
```

```
Available Balance: Rs.7000.0
```

```
===== TRANSACTION 1 =====
```

```
Payment successful!
Paid to       : arun@upi
Amount paid   : Rs.2000.0
Transaction 1 completed.
```

```
===== TRANSACTION 2 =====
```

```
Transaction failed: Invalid UPI ID: invalidupi
Transaction 2 completed.
```

```
===== TRANSACTION 3 =====
```

```
Transaction failed: Payment amount must be greater than zero.
Transaction 3 completed.
```

```
===== TRANSACTION 4 =====
```

```
Transaction failed: Insufficient wallet balance.
Transaction 4 completed.
```

```
===== WALLET DETAILS =====
```

```
User Name      : Rahul
Mobile Number  : 9876543210
UPI ID        : rahul@upi
Balance       : Rs.5000.0
```

```
===== FINAL BALANCE =====
```

```
Available Balance: Rs.5000.0
```

Result

The **UPI Payment and Digital Wallet System** was successfully implemented in Java using an **interface, classes, encapsulation, constructors, custom exceptions, String validation, and try-catch-finally exception handling.**

The program successfully performs **wallet creation, money addition, balance checking, UPI payment, invalid UPI detection, invalid amount detection, and insufficient balance detection.** The final wallet balance after the successful transaction is **Rs.5000.**